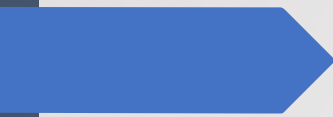




BSM422 Bilgisayarla Görme ve Görüntüleme Teknikleri

Dr. Öğr. Üyesi Mehmet Zahid YILDIRIM



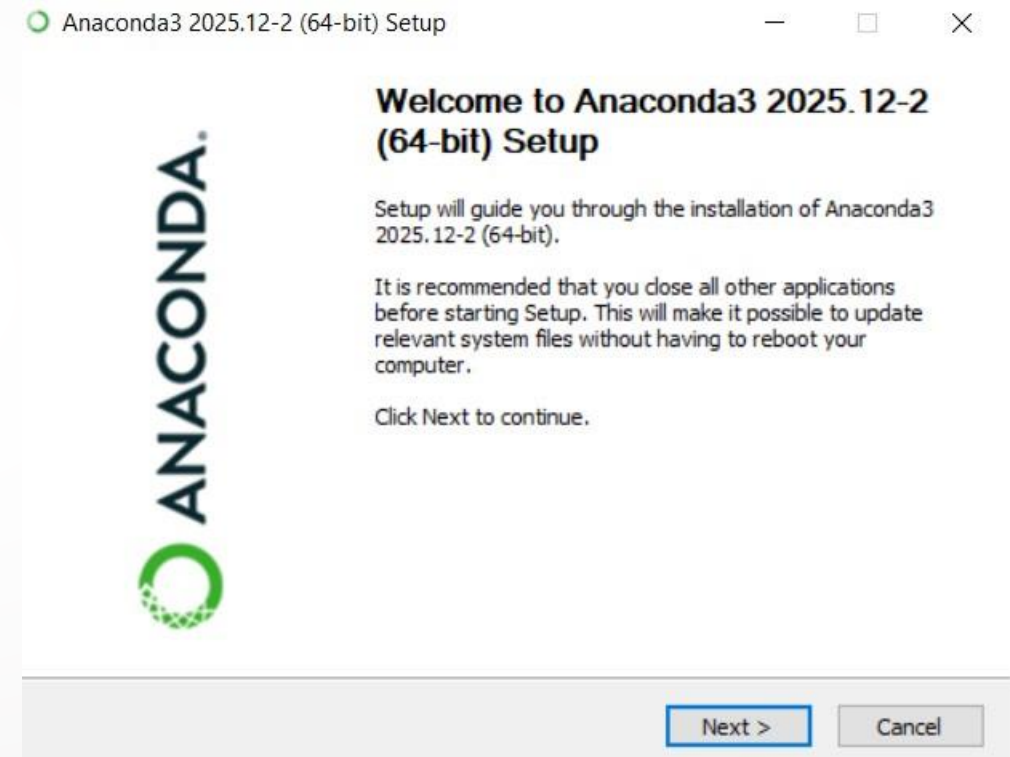
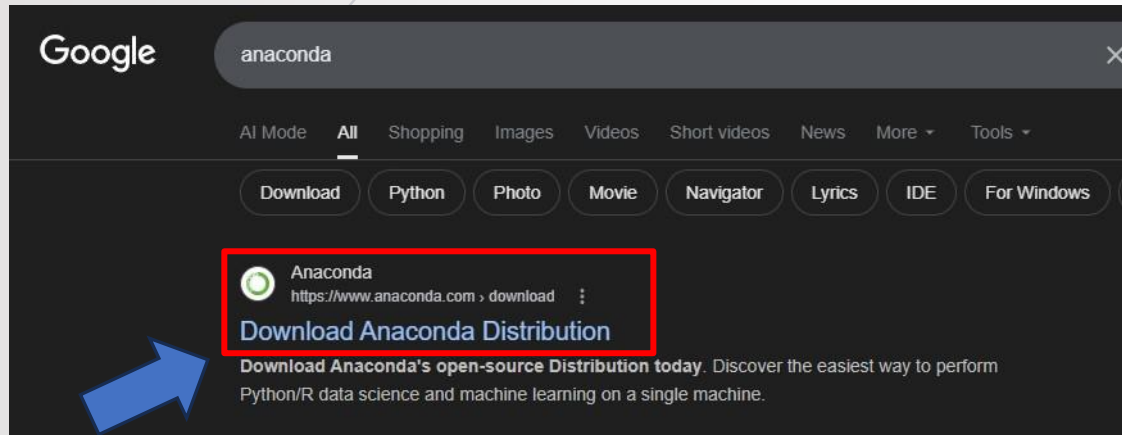
Bu ders;

Python Programlama Dili;

- Kurulumlar
- Jupyter Notebook
- Python Söz Dizimi
 - Veri Tipleri
 - Koşullu İfadeler
 - Döngüler
 - Fonksiyonlar
- Python Kütüphaneleri
 - Numpy, Pandas, Matplotlib, OS

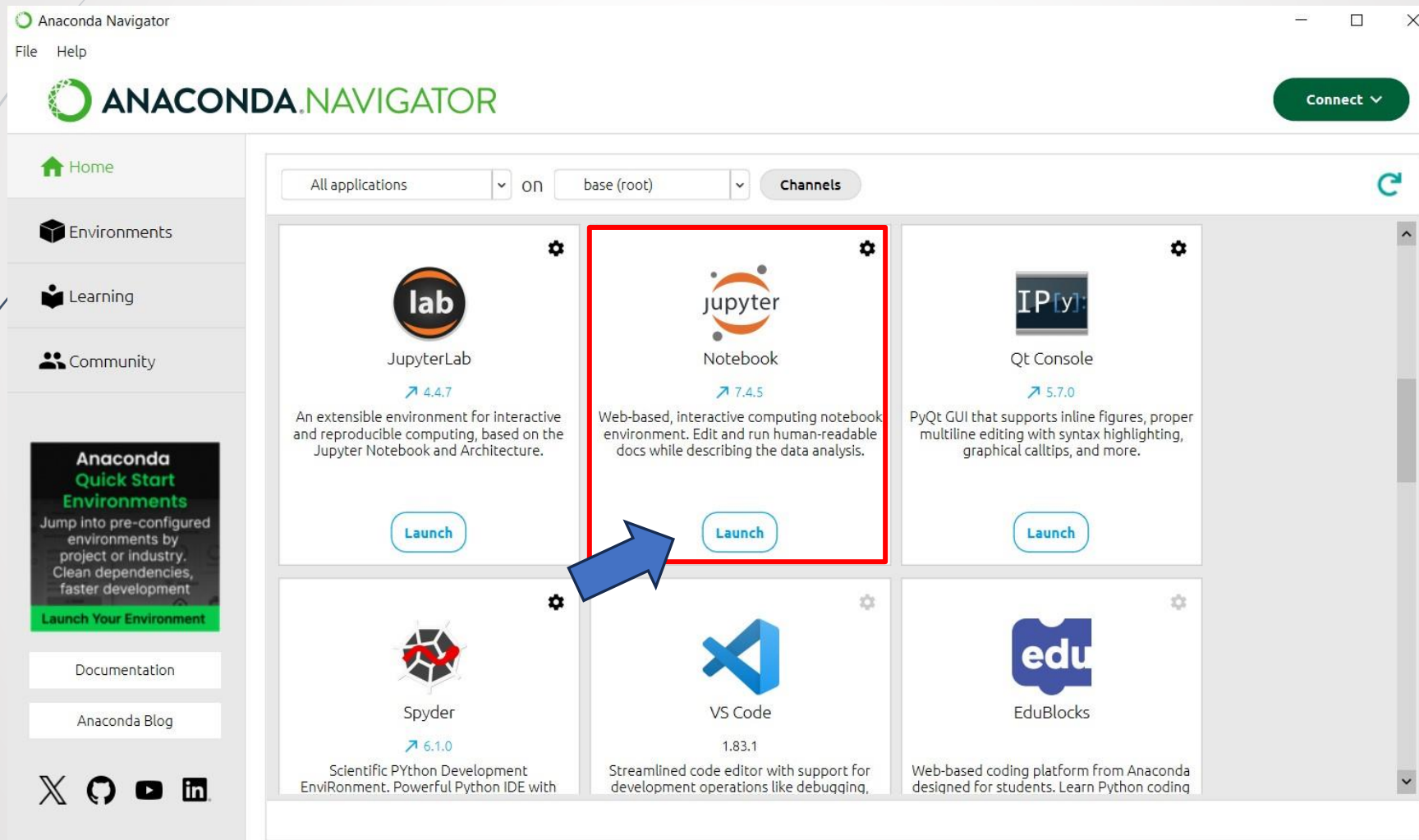
Python

Anaconda Kurulumu



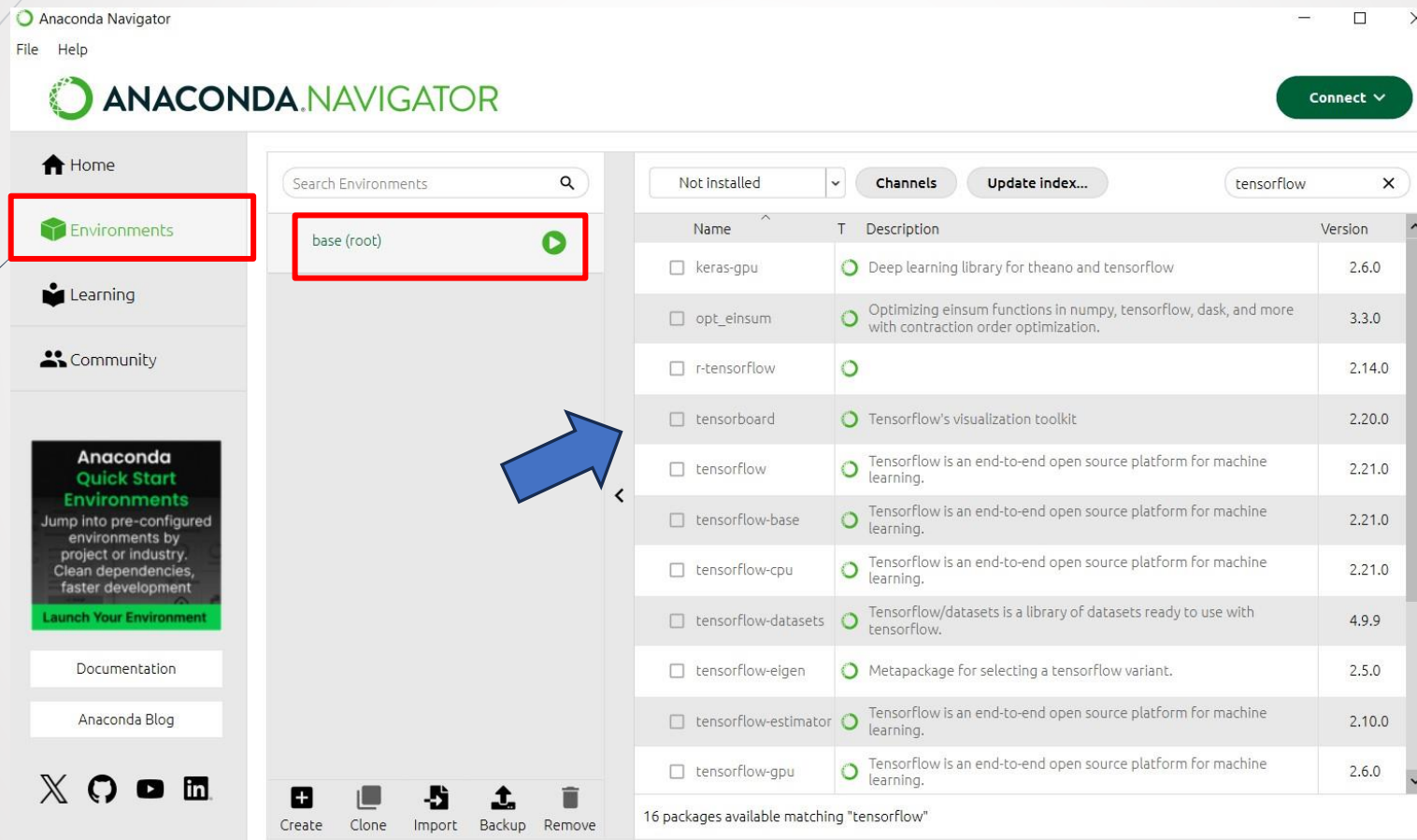
Python

Anaconda Navigator



Python

Anaconda Navigator - Environments



Anaconda Navigator

File Help

ANACONDA.NAVIGATOR

Connect

Home

Environments

Learning

Community

Anaconda Quick Start Environments

Jump into pre-configured environments by project or industry. Clean dependencies, faster development.

Launch Your Environment

Documentation

Anaconda Blog

Create Clone Import Backup Remove

Search Environments

base (root)

Not installed

Channels

Update index...

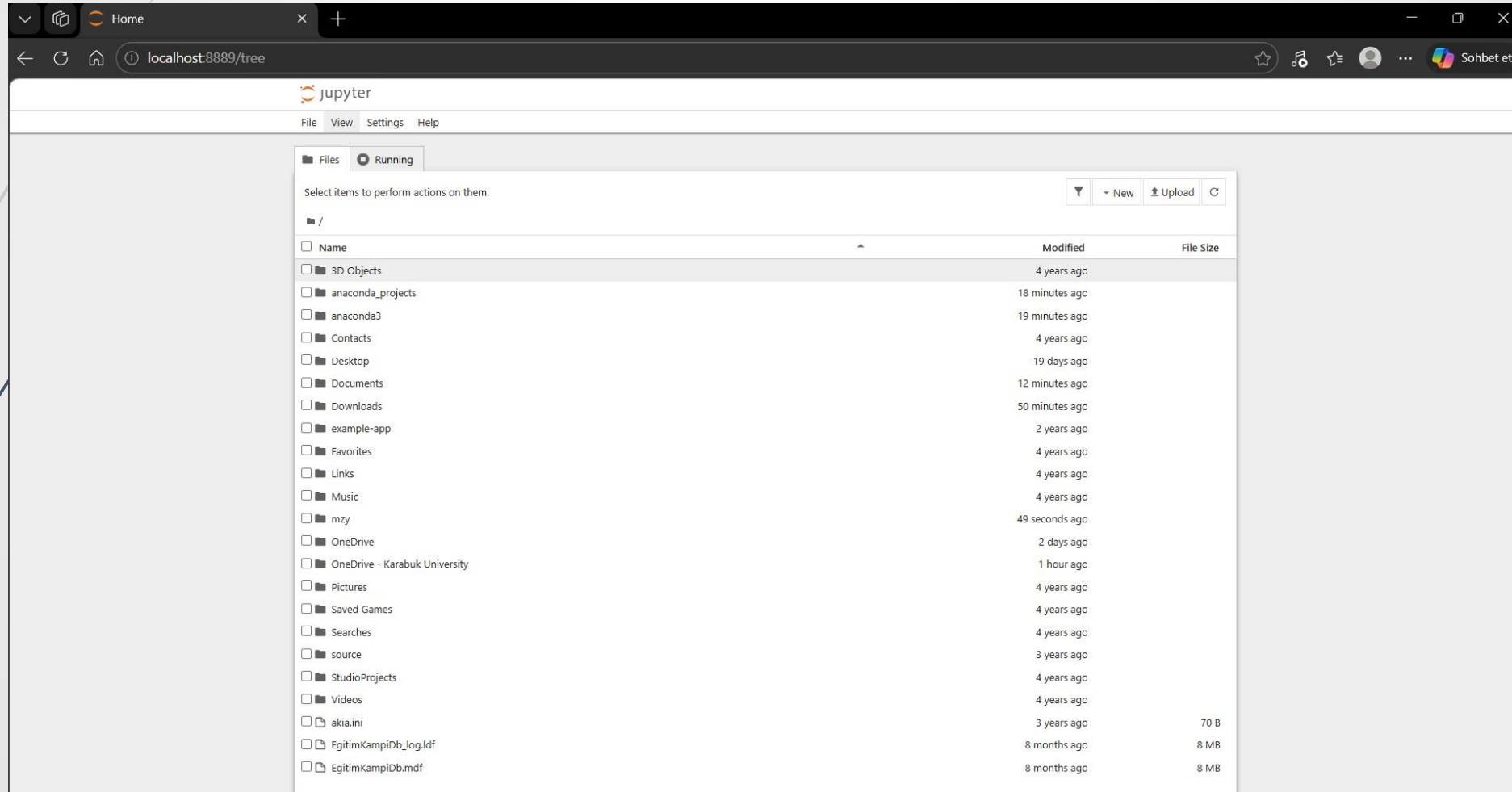
tensorflow

Name	T	Description	Version
☐ keras-gpu	✓	Deep learning library for theano and tensorflow	2.6.0
☐ opt_einsum	✓	Optimizing einsum functions in numpy, tensorflow, dask, and more with contraction order optimization.	3.3.0
☐ r-tensorflow	✓		2.14.0
☐ tensorboard	✓	Tensorflow's visualization toolkit	2.20.0
☐ tensorflow	✓	Tensorflow is an end-to-end open source platform for machine learning.	2.21.0
☐ tensorflow-base	✓	Tensorflow is an end-to-end open source platform for machine learning.	2.21.0
☐ tensorflow-cpu	✓	Tensorflow is an end-to-end open source platform for machine learning.	2.21.0
☐ tensorflow-datasets	✓	Tensorflow/datasets is a library of datasets ready to use with tensorflow.	4.9.9
☐ tensorflow-eigen	✓	Metapackage for selecting a tensorflow variant.	2.5.0
☐ tensorflow-estimator	✓	Tensorflow is an end-to-end open source platform for machine learning.	2.10.0
☐ tensorflow-gpu	✓	Tensorflow is an end-to-end open source platform for machine learning.	2.6.0

16 packages available matching "tensorflow"

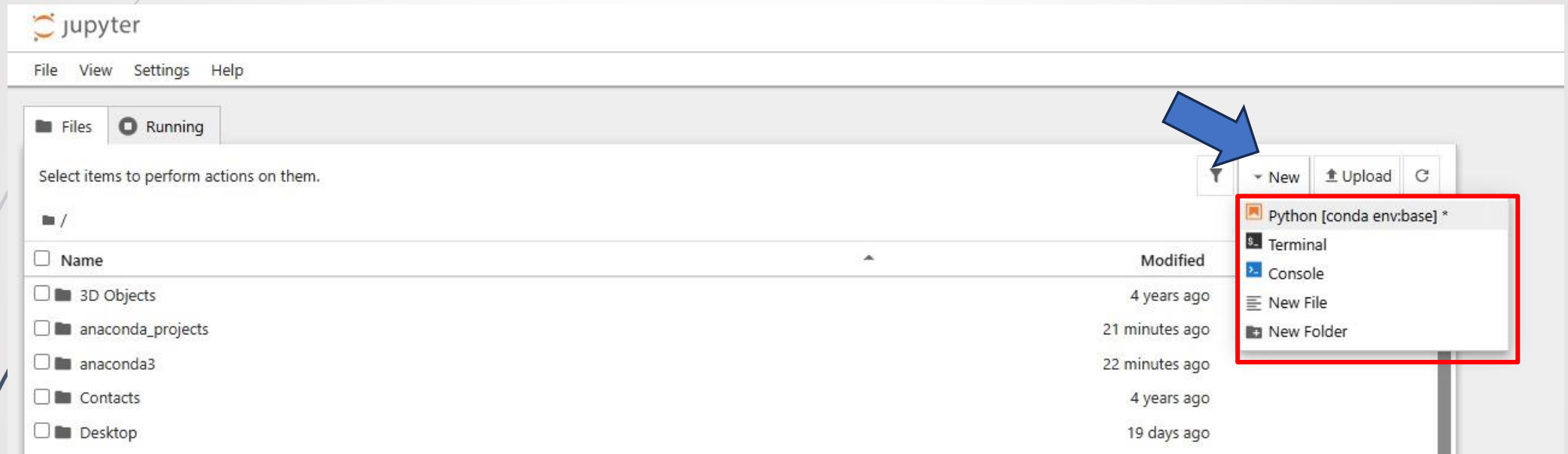
Python

Jupyter Notebook



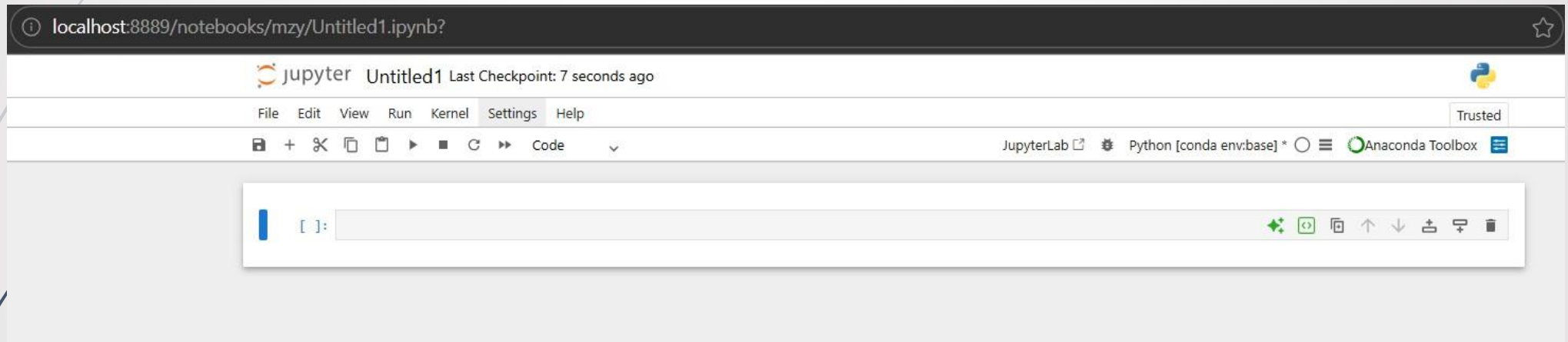
Python

Jupyter Notebook – Yeni Python Projesi



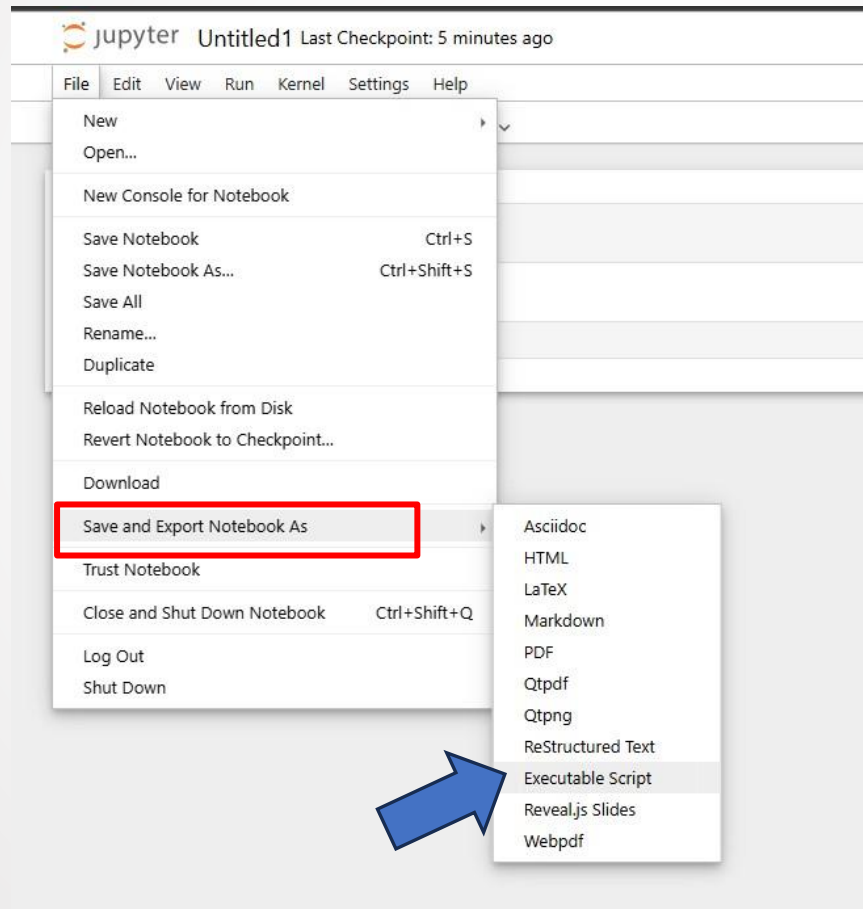
Python

Jupyter Notebook – Kod Ekranı



Python

Jupyter Notebook – Export (.py)



Python

Açıklama Satırları

```
[1]: print("hello world")
```

hello world

```
# Açıklama Satırı
```

```
"""
```

```
Karabük Üniversitesi  
Bilgisayar Mühendisliği  
Bilgisayarla Görü Dersi
```

```
"""
```

Python

Değişkenler

```
[2]: # Değişkenler
tam_sayi = 42
ondalikli_sayi = 55.78
string = "merhaba"

print(tam_sayi)

print(ondalikli_sayi)

print(string)

print(tam_sayi, " - ", ondalikli_sayi, " - ", string)
```

42
55.78
merhaba
42 - 55.78 - merhaba

Python

Dört İşlem

```
[3]: # Dört İşlem
a = 15.2
b = 7

toplam = a + b
fark = a - b
carpim = a * b
bolum = a / b

print("TOPLAM: ", toplam)

print("TOPLAM: {} FARK: {}".format(toplam, fark))

print("CARPIM: %.1f BOLUM: %.2f" % (carpim, bolum))
```

TOPLAM: 22.2
TOPLAM: 22.2 FARK: 8.2
CARPIM: 106.4 BOLUM: 2.17

Python

Değişkenler Arası Dönüşümler

```
[4]: # Değişkenler Arası Dönüşümler  
i = 42  
f = 34.78  
  
print(i, f)  
  
float_i = float(i)  
int_f = int(f)  
  
print(float_i, int_f)
```

42 34.78
42.0 34

Python

Listeler

```
[5]: # Listeler

sayilar = [10, 20, 30, 40, 50]
print(sayilar)

karisik_liste = [25, "Python", 3.14, True]
print(karisik_liste)
print("-----")

meyveler = ["elma", "armut", "muz", "çilek", "portakal"]
print(len(meyveler)) # eleman sayısı
print(meyveler[0])   # ilk eleman
print(meyveler[-1])  # son eleman
print(meyveler[:3])  # ilk üç eleman
print(meyveler[1:4]) # 2 ve 4. eleman arası
print("-----")
```

```
[10, 20, 30, 40, 50]
[25, 'Python', 3.14, True]
-----
5
elma
portakal
['elma', 'armut', 'muz']
['armut', 'muz', 'çilek']
-----
```

Python

Listeler

```
liste = [1, 2, 3]
liste.append(4)      # sona ekle
print(liste)
liste.insert(1, 10)  # indexe ekle
print(liste)
print("-----")

sayilar = [10, 20, 30, 40]
sayilar.remove(20)   # değere göre sil
print(sayilar)
sayilar.pop()        # sondan sil
print(sayilar)
del sayilar[0]        # indexe göre sil
print(sayilar)
print("-----")

sayilar = [15, 7, 21, 32, 4, 5, 18]
sayilar.reverse()
print(sayilar)
sayilar.sort()
print(sayilar)
```



```
[1, 2, 3, 4]
[1, 10, 2, 3, 4]
-----
[10, 30, 40]
[10, 30]
[30]
-----
[18, 5, 4, 32, 21, 7, 15]
[4, 5, 7, 15, 18, 21, 32]
```

Python

Tuple

```
[6]: #Tuple
      # Değiştirilemez (salt okunur) veri tipidir.

      tuple_ = (1, 2, 3, 4, 4, 5)
      print(tuple_[0])          #ilk eleman
      print(tuple_[2:])         #2.indexten sonraki elemanlar
      print(tuple_.count(4))    #eleman sayısı
      print("-----")
      tuple_rgb = (120, 35, 79)
      r, g, b = tuple_rgb
      print(r, g, b)
```

1
(3, 4, 4, 5)
2

120 35 79

Python

Deque

```
[7]: #Deque
from collections import deque
dq = deque(maxlen=3)
dq.append(1)
dq.append(2)
dq.appendleft(3)
print(dq)
print("-----")
dq.clear()
print(dq)
```

→

```
deque([3, 1, 2], maxlen=3)
-----
deque([], maxlen=3)
```

Python

Dictionary

```
[8]: #Dictionary
      #Key ve Value çiftlerinden oluşan veri tipidir.

      dictionary = {"Karabük": 78,
                    "Konya" : 42,
                    "Samsun" : 55}
      print(dictionary)

      print(dictionary.keys())

      print(dictionary.values())

      print(dictionary["Karabük"])
```

→

```
{'Karabük': 78, 'Konya': 42, 'Samsun': 55}
dict_keys(['Karabük', 'Konya', 'Samsun'])
dict_values([78, 42, 55])
78
```

Python

Koşullu İfadeler

```
[9]: #Koşullu İfadeler (if-else)
sayi1 = 42.0
sayi2 = 78.0

if sayi1 < sayi2:
    print("sayi1 küçüktür sayi2")
elif sayi1 > sayi2:
    print("sayi1 büyüktür sayi2")
else:
    print("sayi1 eşittir sayi2")

print("-----")

liste = [1, 2, 3, 4, 5]
deger = 32
if deger in liste:
    print("{} sayısı listenin içerisindedir.".format(deger))
else:
    print("{} sayısı listenin içerisinde değildir.".format(deger))

print("-----")
```

sayi1 küçüktür sayi2

32 sayısı listenin içerisinde değildir.

Python

Koşullu İfadeler

```
dictionary = {"Karabük": 78, "Konya" : 42, "Samsun" : 55}
keys = dictionary.keys()
deger = "Karabük"
if deger in keys:
    print("evet")
else:
    print("hayır")

print("-----")

bool1 = True
bool2 = False

if bool1 and bool2:
    print("and doğru")

if bool1 or bool2:
    print("or doğru")
```

evet

or doğru

Python

Döngüler

```
[10]: #Döngüler (for - while)
meyveler = ["elma", "muz", "çilek"]

for meyve in meyveler:
    print(meyve)

print("-----")

for i in range(5):
    print(i)

print("-----")

for i in range(2, 10, 2):
    print(i)
```

elma
muz
çilek

0
1
2
3
4

2
4
6
8

Python

Döngüler

```
print("-----")

toplam = 0
for i in range(1, 6):
    toplam += i

print(toplam)

print("-----")

i = 0
while i < 5:
    print(i)
    i += 1
```



15

0

1

2

3

4

Python

Fonksiyonlar

```
[11]: #Fonksiyonlar
def yazdir():
    print("Merhaba!")

yazdir()

print("-----")

def yazdir(str):
    print(str)

yazdir("Karabük")

print("-----")

def topla(a, b):
    print(a + b)

topla(3, 5)
```

Merhaba!

Karabük

8

Python

Fonksiyonlar

```
def toplama(a, b):  
    return a + b  
  
sonuc = toplama(4, 6)  
print(sonuc)  
  
print("-----")  
  
def daire_alani(r, pi = 3.14):  
    return pi * r * r  
  
sonuc = daire_alani(4)  
sonuc2 = daire_alani(4, 3)  
print(sonuc, sonuc2)
```

10

50.24 48

Python

Fonksiyonlar

```
def islem(a, b):
    toplam = a + b
    carpim = a * b
    return toplam, carpim

t, c = islem(3, 4)
print(t, c)

print("-----")

def ortalama(liste):
    return sum(liste) / len(liste)

notlar = [70, 80, 90]
print(ortalama(notlar))

print("-----")
#-----Lambda Fonksiyon-----

kare = lambda x: x**2
print(kare(5))
```

7 12

80.0

25

Python

Yield

```
[12]: #Yield
      """
      iterasyon: Bir veri yapısının elemanlarını tek tek dolaşma işlemine denir.
      generator: Elemanları hafızada tutmadan, ihtiyaç oldukça üreten özel iterator'dür.
      yield     : Fonksiyonu generator'e dönüştürür.
      """

      liste = [1, 2, 3]
      for i in liste:
          print(i)
```

1
2
3

Python

Yield

```
generator = (x for x in range(1, 4))
for i in generator:
    print(i)

print("-----")

def createGenerator():
    liste = range(1,4)
    for i in liste:
        yield i

generator = createGenerator()
for i in generator:
    print(i)
```

1
2
3

1
2
3

Python - Kütüphaneler

Numpy

```
[13]: #Numpy (Matris İşlemleri)
      """
      - NumPy, Python'da sayısal hesaplama, matris işlemleri ve bilimsel programlama için kullanılan en temel kütüphanedir.
      - NumPy → Numerical Python.
      - Listelere göre daha hızlıdır, daha az bellek kullanır.
      """

      import numpy as np
      # Array oluşturma
      a = np.array([1, 2, 3, 4])
      print(a)
      print("-----")

      # Matris oluşturma
      matris = np.array([
          [1, 2, 3],
          [4, 5, 6]
      ])
      print(matris)
      print("-----")
```

→

```
[1 2 3 4]
-----
[[1 2 3]
 [4 5 6]]
-----
```

Python - Kütüphaneler

Numpy

```
print(matris.shape)    # satır x sütun bilgisi
print(matris.ndim)     # boyut sayısı
print(matris.size)     # toplam eleman sayısı
print("-----")

arr = np.zeros((3,3))   # sıfırlardan oluşan matris
print(arr)
print("-----")
arr2 = np.ones((2,4))   # birlerden oluşan matris
print(arr2)
print("-----")
arr3 = np.arange(0,10,2) # belirli aralıklar ile matris
print(arr3)
print("-----")
arr4 = np.linspace(0,1,5) # eşit aralıklı matris
print(arr4)
print("-----")
```

```
(2, 3)
2
6
-----
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
-----
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]]
-----
[0 2 4 6 8]
-----
[0.    0.25 0.5   0.75 1.   ]
-----
```

Python - Kütüphaneler

Numpy

```
# Matematiksel işlemler  
a = np.array([1,2,3])  
b = np.array([4,5,6])
```

```
print(a + b)  
print(a * 2)  
print(a ** 2)  
print("-----")
```

```
# indexing  
matris = np.array([  
    [1,2,3],  
    [4,5,6]  
])
```

```
print(matris[0,1])  
print("-----")
```

```
[5 7 9]  
[2 4 6]  
[1 4 9]  
-----  
2  
-----
```

Python - Kütüphaneler

Numpy

```
a = np.array([1,2,3,4,5])

print(np.mean(a))    # ortalama
print(np.sum(a))     # toplam
print(np.max(a))     # maksimum
print(np.min(a))     # minimum
print("-----")

# Boyut değiştirme
a = np.arange(12)
print(a)
b = a.reshape(3,4)
print(b)
print("-----")
```

```
3.0
15
5
1
-----
[ 0  1  2  3  4  5  6  7  8  9 10 11]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
-----
```

Python - Kütüphaneler

Numpy

```
# Rasgele sayı üretme
rasgeleDizi = np.random.rand(3,3)           # 0-1 arası
print(rasgeleDizi)
print("-----")

rasgeleDizi2 = np.random.randint(0,10,(3,3)) # 0-10 arası tam sayı
print(rasgeleDizi2)
```

```
[[0.33723923 0.64526027 0.87102712]
 [0.76405017 0.93351726 0.65598427]
 [0.4835664  0.94277083 0.95234506]]
```

```
-----
[[1 2 9]
 [9 7 6]
 [2 0 9]]
```

Python - Kütüphaneler

Pandas

```
[1]: #Pandas
      """
      - Python'da veri analizi ve veri işleme için en yaygın kullanılan kütüphanelerden biridir.
      - Özellikle tablo (Excel gibi) verileriyle çalışmayı çok kolaylaştırır.
      - Veri analizi, veri temizleme, makine öğrenmesi öncesi veri hazırlama,
        Excel/Csv işlemleri ve Zaman serisi analizinde kullanılır.
      """

      import pandas as pd

      """
      İki temel veri yapısı vardır:
      - Series → Tek boyutlu veri (liste gibi)
      - DataFrame → İki boyutlu veri (tablo gibi)
      """
```

Python - Kütüphaneler

Pandas

```
# Series Örneği
veri = [10, 20, 30, 40]
seri = pd.Series(veri)

print(seri)

# Data Frame Örneği
data = {
    "Ad": ["Ali", "Ayşe", "Mehmet"],
    "Yaş": [25, 30, 22],
    "Şehir": ["Ankara", "İstanbul", "İzmir"]
}

df = pd.DataFrame(data)
print(df)
```

0 10
1 20
2 30
3 40

	Ad	Yaş	Şehir
0	Ali	25	Ankara
1	Ayşe	30	İstanbul
2	Mehmet	22	İzmir

Python - Kütüphaneler

Pandas

```
# Veri Okuma (Excel/Csv)
df = pd.read_csv("veri.csv")
df = pd.read_excel("veri.xlsx")
```

Python - Kütüphaneler

Pandas

```
# Veri İnceleme
data = {
    "Ad": ["Ali", "Ayşe", "Mehmet", "Beyza", "Talha", "Ahmet", "Elif"],
    "Yaş": [25, 30, 22, 10, 18, 42, 55],
    "Şehir": ["Ankara", "İstanbul", "İzmir", "Konya", "Samsun", "Karabük", "Bursa"]
}
```

```
df = pd.DataFrame(data)
```

```
df.head()      # İlk 5 satır
```

```
df.tail()      # Son 5 satır
```

```
df.info()      # Veri tipi bilgileri
```

```
df.describe()  # İstatistiksel özet
```

	Ad	Yaş	Şehir
0	Ali	25	Ankara
1	Ayşe	30	İstanbul
2	Mehmet	22	İzmir
3	Beyza	10	Konya
4	Talha	18	Samsun

	Ad	Yaş	Şehir
2	Mehmet	22	İzmir
3	Beyza	10	Konya
4	Talha	18	Samsun
5	Ahmet	42	Karabük
6	Elif	55	Bursa

Python - Kütüphaneler

Pandas

```
data = {  
    "Ad": ["Ali", "Ayşe", "Mehmet", "Beyza", "Talha", "Ahmet", "Elif"],  
    "Yaş": [25, 30, 22, 10, 18, 42, 55],  
    "Şehir": ["Ankara", "İstanbul", "İzmir", "Konya", "Samsun", "Karabük", "Bursa"]  
}
```

```
df = pd.DataFrame(data)
```

```
df.head()      # İlk 5 satır
```

```
df.tail()      # Son 5 satır
```

```
df.info()      # Veri tipi bilgileri
```

```
df.describe()  # İstatistiksel özet
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 7 entries, 0 to 6  
Data columns (total 3 columns):  
#   Column  Non-Null Count  Dtype  
---  ---  
0    Ad      7 non-null       object  
1    Yaş      7 non-null       int64  
2    Şehir    7 non-null       object  
dtypes: int64(1), object(2)  
memory usage: 300.0+ bytes
```

	Yaş
count	7.000000
mean	28.857143
std	15.236235
min	10.000000
25%	20.000000
50%	25.000000
75%	36.000000
max	55.000000

Python - Kütüphaneler

Pandas

```
# Yeni Sütun Ekleme
data = {
    "Ad": ["Ali", "Ayşe", "Mehmet"],
    "Yaş": [25, 30, 22],
    "Şehir": ["Ankara", "İstanbul", "İzmir"]
}
df = pd.DataFrame(data)
print(df)

df["Maaş"] = [1000, 3500, 1800]
print(df)
```



	Ad	Yaş	Şehir
0	Ali	25	Ankara
1	Ayşe	30	İstanbul
2	Mehmet	22	İzmir



	Ad	Yaş	Şehir	Maaş
0	Ali	25	Ankara	1000
1	Ayşe	30	İstanbul	3500
2	Mehmet	22	İzmir	1800

Python - Kütüphaneler

Pandas

```
import pandas as pd
import numpy as np

# Eksik Veri Oluşturma
data = {
    "Ad": ["Ali", "Ayşe", "Mehmet", "Zeynep", "Ahmet"],
    "Yaş": [25, np.nan, 22, 30, np.nan],
    "Not": [80, 90, np.nan, 75, 85]
}

df = pd.DataFrame(data)
print(df)
```

	Ad	Yaş	Not
0	Ali	25.0	80.0
1	Ayşe	NaN	90.0
2	Mehmet	22.0	NaN
3	Zeynep	30.0	75.0
4	Ahmet	NaN	85.0

Python - Kütüphaneler

Pandas

Eksik Verilerin Tespiti

```
df.isnull()
```

```
df.isnull().sum()
```

Ad	0
Yaş	2
Not	1

	Ad	Yaş	Not
0	Ali	25.0	80.0
1	Ayşe	NaN	90.0
2	Mehmet	22.0	NaN
3	Zeynep	30.0	75.0
4	Ahmet	NaN	85.0

	Ad	Yaş	Not
0	False	False	False
1	False	True	False
2	False	False	True
3	False	False	False
4	False	True	False

Python - Kütüphaneler

Pandas

```
# Eksik Veri İçeren Satırları Silme  
df_drop = df.dropna()  
print(df_drop)
```

	Ad	Yaş	Not
0	Ali	25.0	80.0
1	Ayşe	NaN	90.0
2	Mehmet	22.0	NaN
3	Zeynep	30.0	75.0
4	Ahmet	NaN	85.0



	Ad	Yaş	Not
0	Ali	25.0	80.0
3	Zeynep	30.0	75.0

Python - Kütüphaneler

Pandas

Eksik Verileri Sıfır (0) ile Doldurma.

```
df_fill = df.fillna(0)
```

```
print(df_fill)
```

Eksik Verileri Ortalama ile Doldurma.

```
df["Yaş"] = df["Yaş"].fillna(df["Yaş"].mean())
```

```
df["Not"] = df["Not"].fillna(df["Not"].mean())
```

```
print(df)
```

	Ad	Yaş	Not
0	Ali	25.0	80.0
1	Ayşe	NaN	90.0
2	Mehmet	22.0	NaN
3	Zeynep	30.0	75.0
4	Ahmet	NaN	85.0

	Ad	Yaş	Not
0	Ali	25.0	80.0
1	Ayşe	0.0	90.0
2	Mehmet	22.0	0.0
3	Zeynep	30.0	75.0
4	Ahmet	0.0	85.0

	Ad	Yaş	Not
0	Ali	25.000000	80.0
1	Ayşe	25.666667	90.0
2	Mehmet	22.000000	82.5
3	Zeynep	30.000000	75.0
4	Ahmet	25.666667	85.0

Python - Kütüphaneler

Pandas

```
# Filtreleme
dictionary = {
    "isim": ["Ali", "Ayşe", "Mehmet", "Murat", "Hilal", "Zeynep"],
    "yas": [25, 30, 22, 18, 39, 19],
    "sehir": ["Ankara", "İstanbul", "İzmir", "Ankara", "Ankara", "Konya"]
}
veri = pd.DataFrame(dictionary)

print(veri)

filtre = veri.yas > 25

print(filtre)

print(veri[filtre])
```

	isim	yas	sehir
0	Ali	25	Ankara
1	Ayşe	30	İstanbul
2	Mehmet	22	İzmir
3	Murat	18	Ankara
4	Hilal	39	Ankara
5	Zeynep	19	Konya

0	False
1	True
2	False
3	False
4	True
5	False

	isim	yas	sehir
1	Ayşe	30	İstanbul
4	Hilal	39	Ankara

Python - Kütüphaneler

Pandas

```
# Ortalama yaşa göre yeni sütun oluşturma.  
ort = veri.yas.mean()  
veri["yas grubu"] = ["Genç" if ort > i else "Yaşlı" for i in veri.yas]  
print(veri)
```

	isim	yas	sehir	yas grubu
0	Ali	25	Ankara	Genç
1	Ayşe	30	İstanbul	Yaşlı
2	Mehmet	22	İzmir	Genç
3	Murat	18	Ankara	Genç
4	Hilal	39	Ankara	Yaşlı
5	Zeynep	19	Konya	Genç

Python - Kütüphaneler

Pandas

```
# Veri Birleştirme
dict_1 = {
    "isim": ["Ali", "Ayşe", "Mehmet"],
    "yas": [25, 30, 22],
    "sehir": ["Ankara", "İstanbul", "İzmir"]
}
dict_2 = {
    "isim": ["Murat", "Hilal", "Zeynep"],
    "yas": [18, 39, 19],
    "sehir": ["Ankara", "Ankara", "Konya"]
}
```

```
df_1 = pd.DataFrame(dict_1)
df_2 = pd.DataFrame(dict_2)
```

```
dikey_birlestirme = pd.concat([df_1, df_2], axis = 0)
print(dikey_birlestirme)
```

```
yatay_birlestirme = pd.concat([df_1, df_2], axis = 1)
print(yatay_birlestirme)
```



isim	yas	sehir
Ali	25	Ankara
Ayşe	30	İstanbul
Mehmet	22	İzmir
Murat	18	Ankara
Hilal	39	Ankara
Zeynep	19	Konya



isim	yas	sehir	isim	yas	sehir
Ali	25	Ankara	Murat	18	Ankara
Ayşe	30	İstanbul	Hilal	39	Ankara
Mehmet	22	İzmir	Zeynep	19	Konya

Python - Kütüphaneler

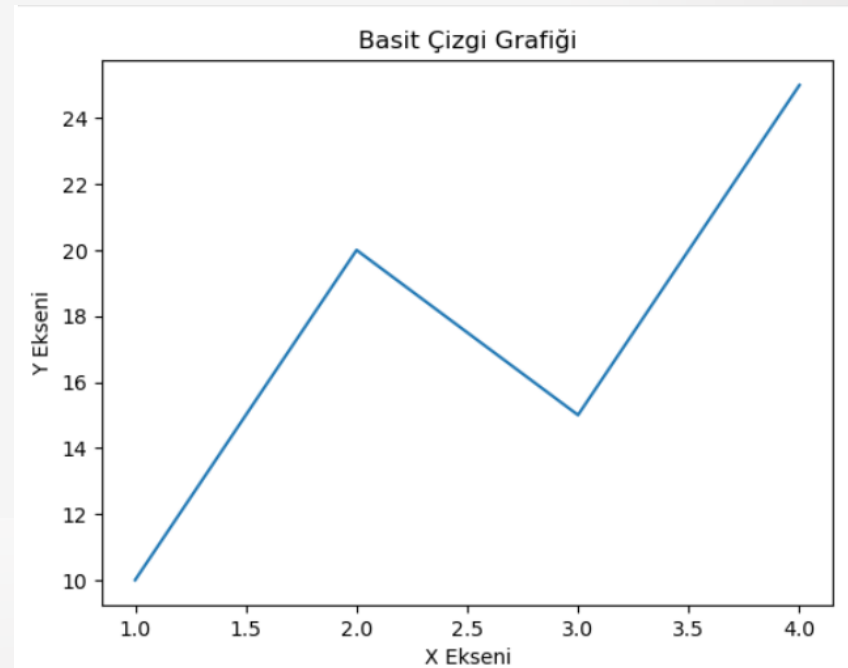
Matplotlib

```
[1]: #Matplotlib
     """
     - Veri görselleştirme için kullanılan en temel ve güçlü kütüphanelerden biridir.
     - Grafik çizmek için kullanılır.
     - Makine öğrenmesi ve veri analizinde çok yaygın kullanılır.
     """

     import matplotlib.pyplot as plt

     # Basit çizgi grafik
     x = [1, 2, 3, 4]
     y = [10, 20, 15, 25]

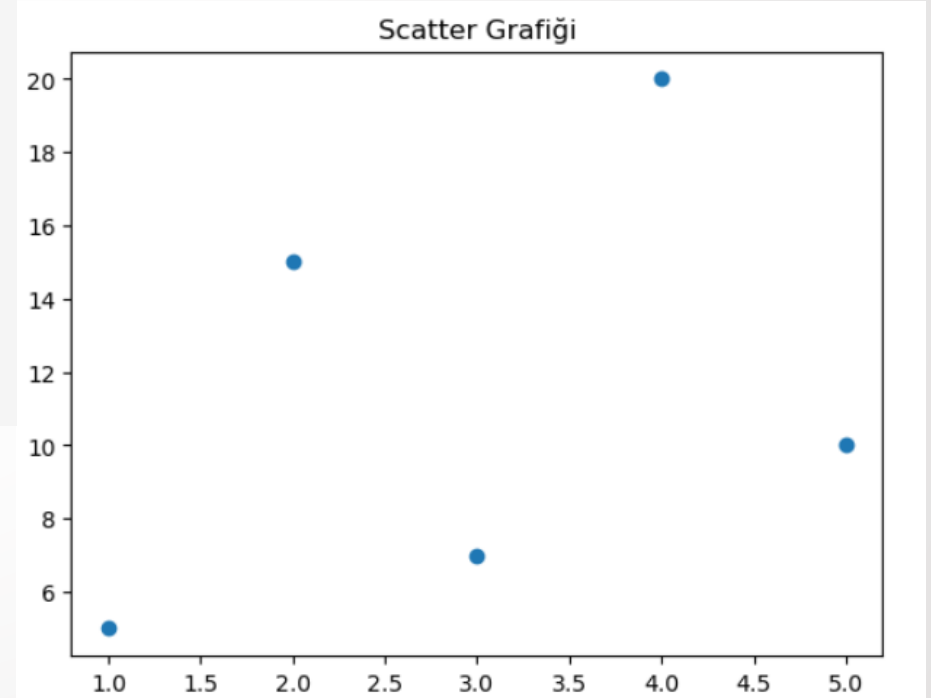
     plt.plot(x, y)
     plt.title("Basit Çizgi Grafiği")
     plt.xlabel("X Eksen")
     plt.ylabel("Y Eksen")
     plt.show()
```



Python - Kütüphaneler

Matplotlib

```
# Scatter (Dağılım) grafiği  
x = [1, 2, 3, 4, 5]  
y = [5, 15, 7, 20, 10]  
  
plt.scatter(x, y)  
plt.title("Scatter Grafiği")  
plt.show()
```

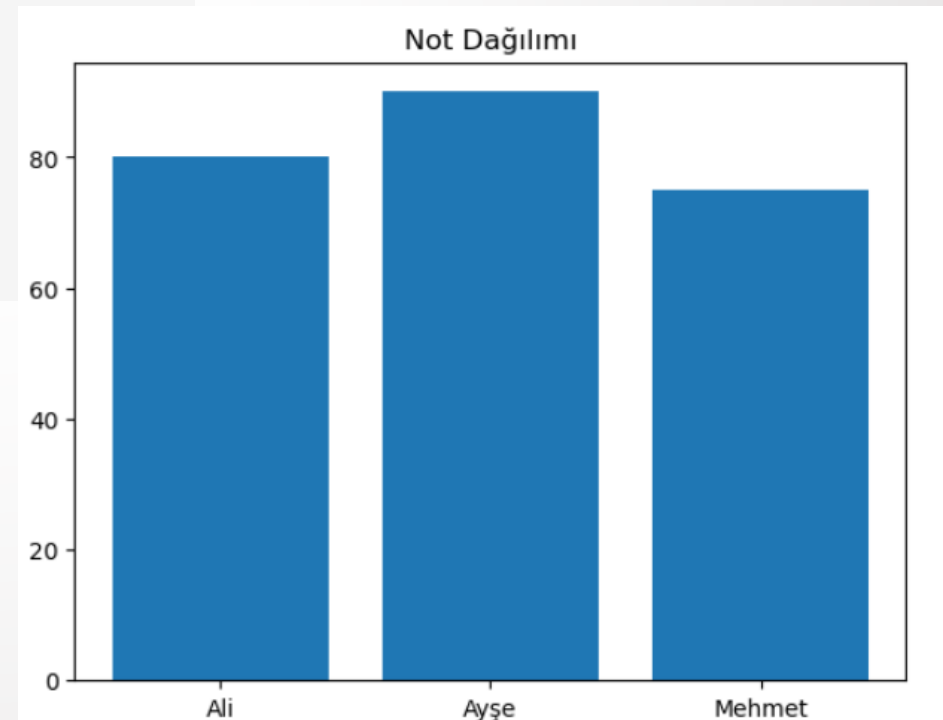


Python - Kütüphaneler

Matplotlib

```
# Bar (Sütun) grafiği  
isimler = ["Ali", "Ayşe", "Mehmet"]  
notlar = [80, 90, 75]
```

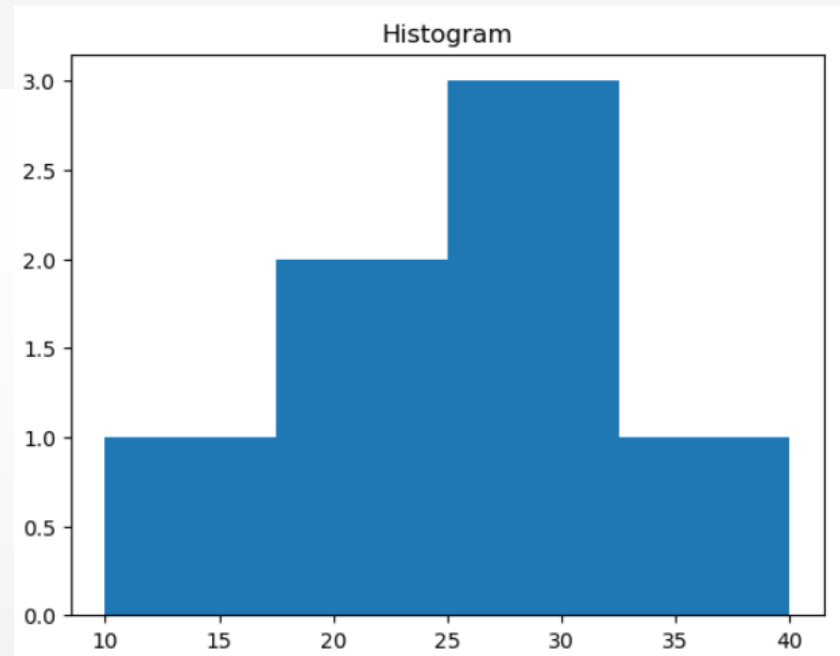
```
plt.bar(isimler, notlar)  
plt.title("Not Dağılımı")  
plt.show()
```



Python - Kütüphaneler

Matplotlib

```
# Histogram  
veri = [10, 20, 20, 30, 30, 30, 40]  
  
plt.hist(veri, bins=4) # bins --> veriyi kaç aralığa (sınıfa) böleceğini belirler.  
plt.title("Histogram")  
plt.show()
```



Python - Kütüphaneler

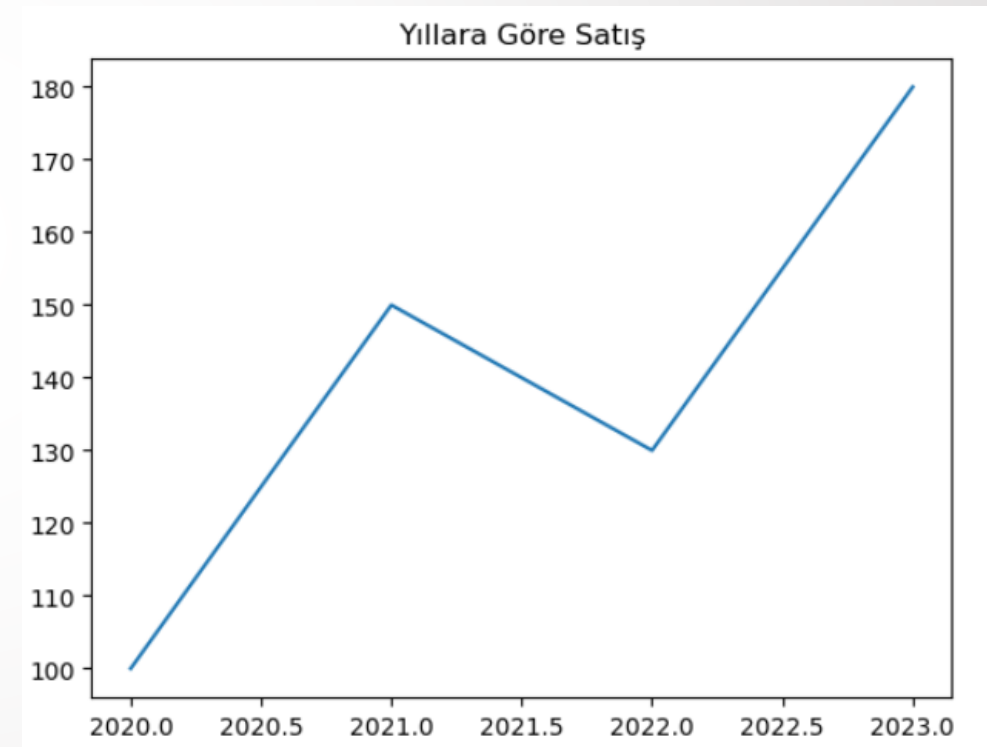
Matplotlib

```
# Pandas ile kullanım
import pandas as pd
import matplotlib.pyplot as plt

data = {
    "Yıl": [2020, 2021, 2022, 2023],
    "Satış": [100, 150, 130, 180]
}

df = pd.DataFrame(data)

plt.plot(df["Yıl"], df["Satış"])
plt.title("Yıllara Göre Satış")
plt.show()
```

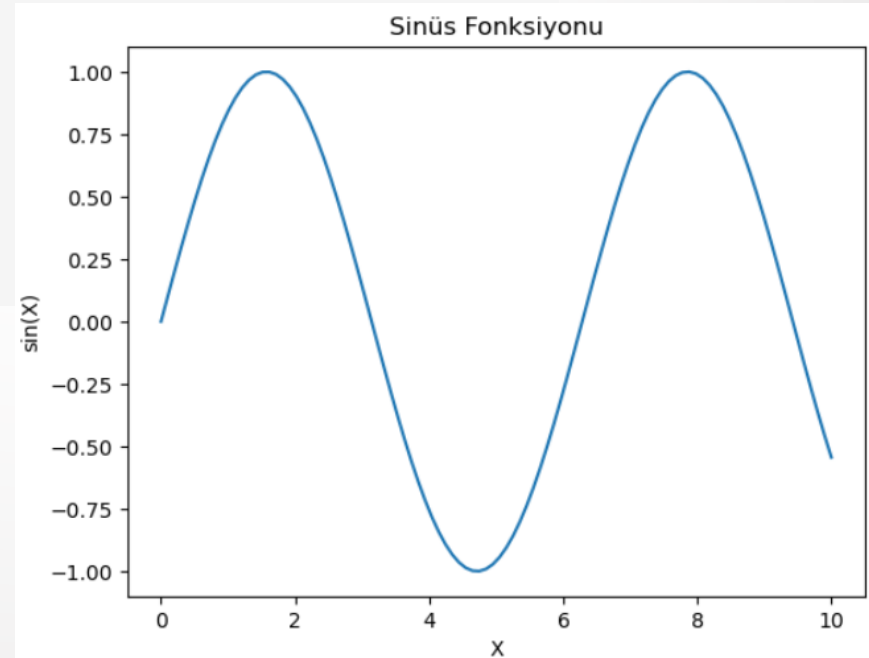


Python - Kütüphaneler

Matplotlib

```
# Numpy ile kullanım
import numpy as np
import matplotlib.pyplot as plt

x = np.linspace(0, 10, 100) # 0-1 arasında eşit aralıkla 100 sayı
y = np.sin(x)
plt.plot(x, y)
plt.title("Sinüs Fonksiyonu")
plt.xlabel("X")
plt.ylabel("sin(X)")
plt.show()
```



Python - Kütüphaneler

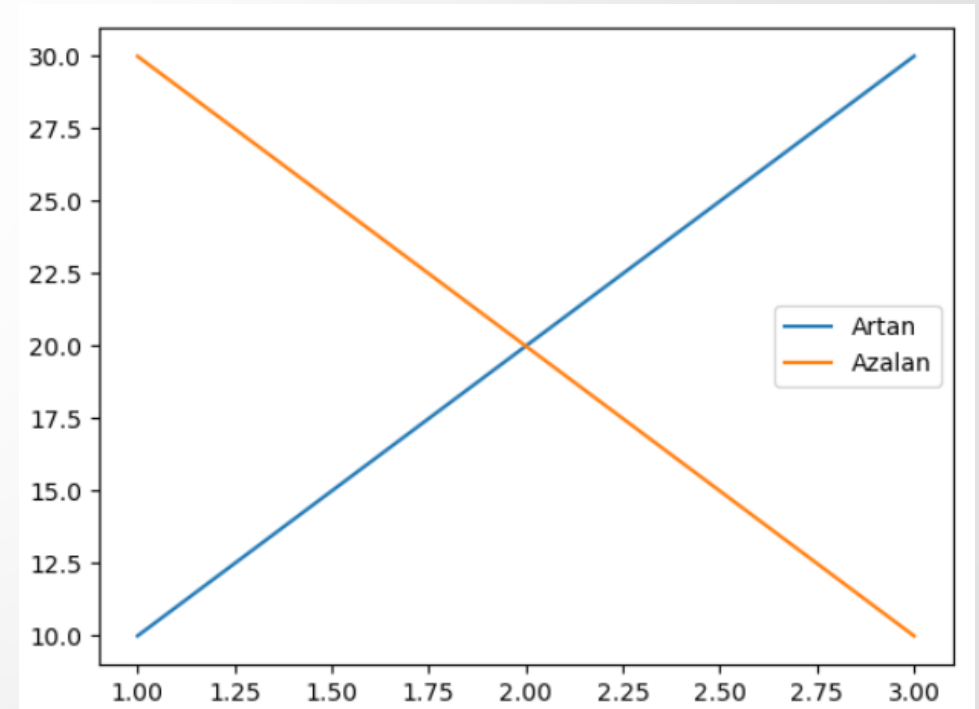
Matplotlib

```
# Birden fazla grafik
x = [1, 2, 3]

y1 = [10, 20, 30]
y2 = [30, 20, 10]

plt.plot(x, y1, label="Artan")
plt.plot(x, y2, label="Azalan")

plt.legend() # grafiklerin label bilgisini ekleme
plt.show()
```



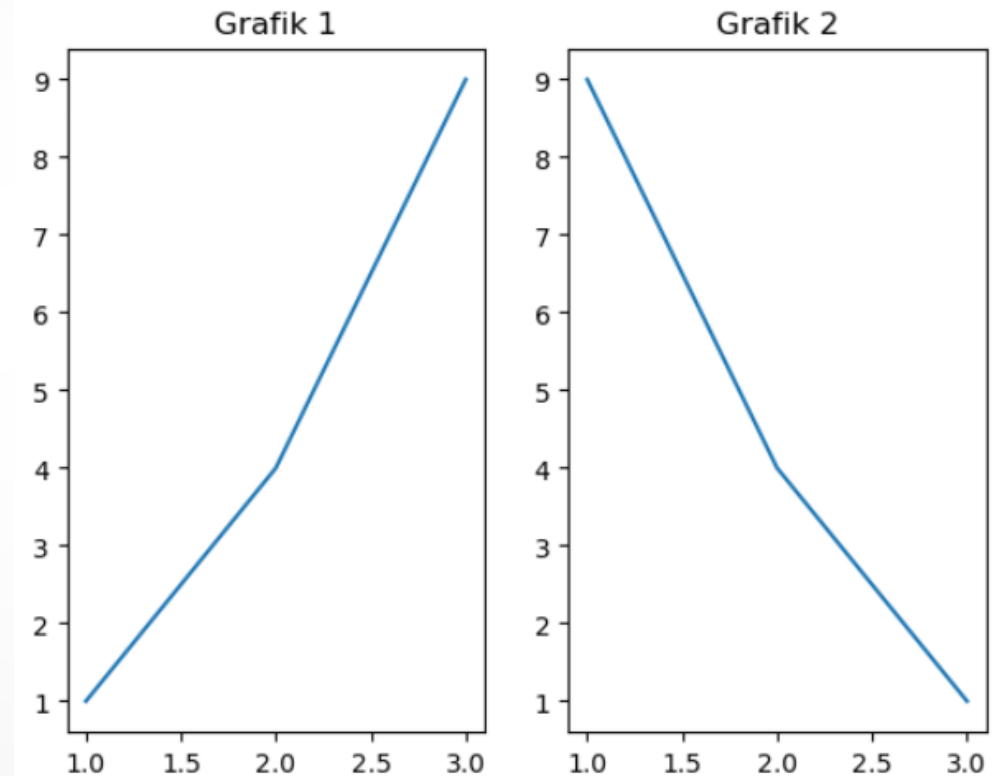
Python - Kütüphaneler

Matplotlib

```
# Alt Grafikler (Subplot)
plt.subplot(1, 2, 1)
plt.plot([1,2,3], [1,4,9])
plt.title("Grafik 1")

plt.subplot(1, 2, 2)
plt.plot([1,2,3], [9,4,1])
plt.title("Grafik 2")

plt.show()
```



Python - Kütüphaneler

Matplotlib

```
plt.subplot(2, 1, 1)
plt.plot([1,2,3], [1,4,9])
plt.title("Grafik 1")
```

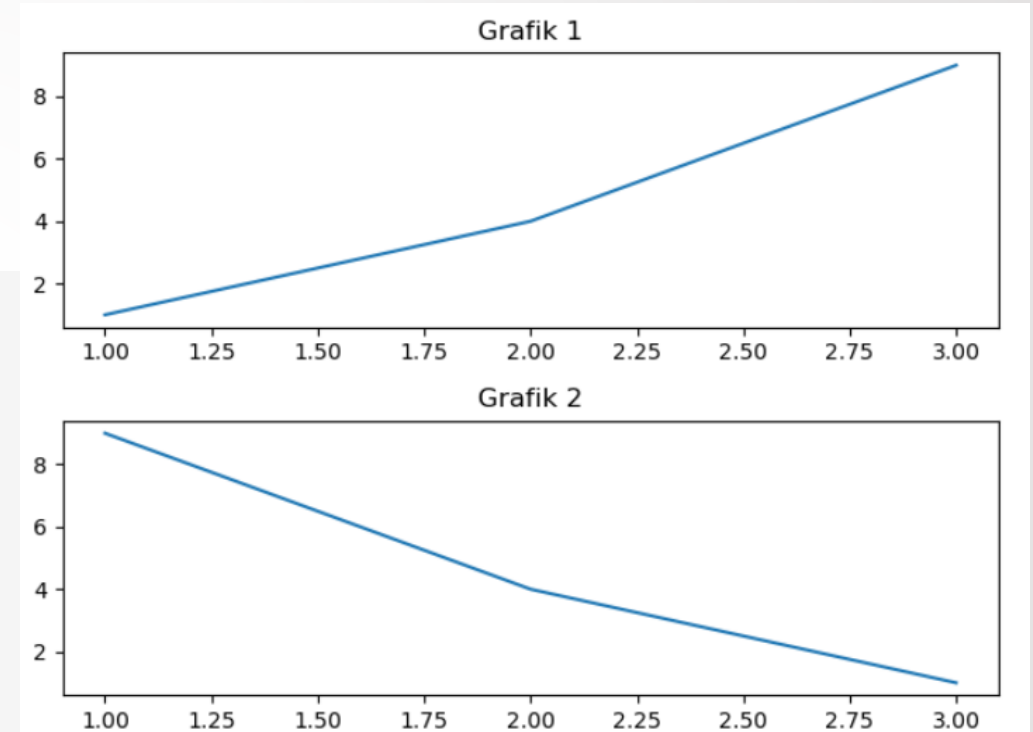
```
plt.subplot(2, 1, 2)
plt.plot([1,2,3], [9,4,1])
plt.title("Grafik 2")
```

otomatik grafiklar arası boşluk ayarlama

```
plt.tight_layout()
```

// manuel ayar için "plt.subplots_adjust(hspace=0.5)" kullanılır.

```
plt.show()
```



Python - Kütüphaneler

OS

```
[1]: #OS
import os

print(os.name)

currentDir = os.getcwd()
print(currentDir)
```

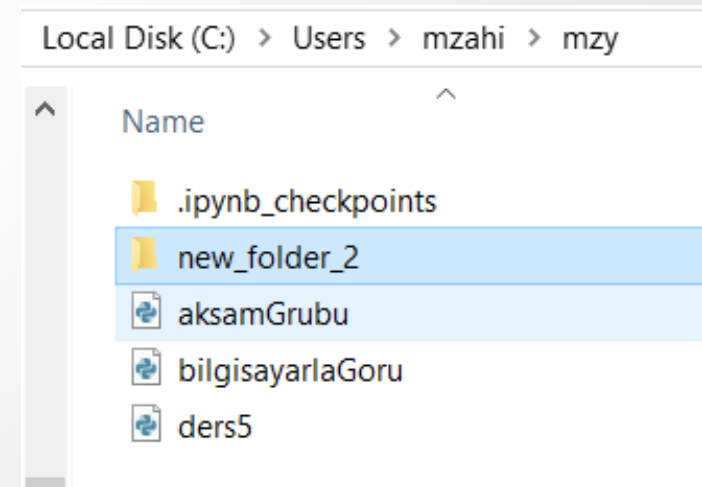
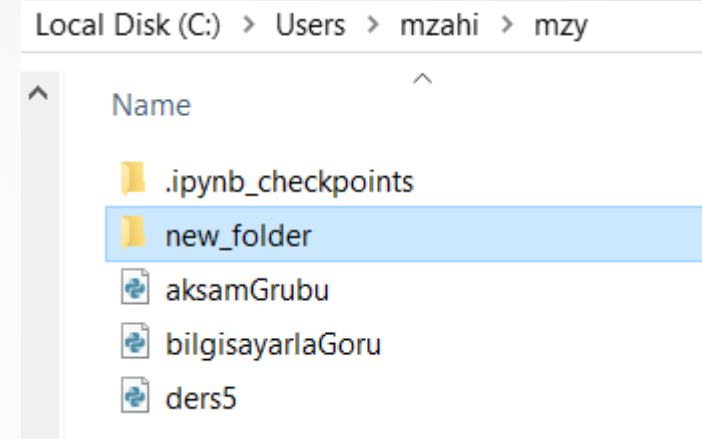
nt

C:\Users\mzahi\mzy

Python - Kütüphaneler

OS

```
# yeni klasör  
folder_name = "new_folder"  
os.mkdir(folder_name)  
  
# klasör ismi değiştirme  
new_folder_name = "new_folder_2"  
os.rename(folder_name, new_folder_name)
```



Python - Kütüphaneler

OS

```
# aktif dizini değiştirme
currentDir = os.getcwd()
print(currentDir)
os.chdir(currentDir + "//new_folder_2")
print(os.getcwd())
```

```
# klasör silme
os.rmdir(new_folder_name)
```

C:\Users\mzahi\mzy
C:\Users\mzahi\mzy\new_folder_2

Python - Kütüphaneler

OS

```
# dosyaları görme  
files = os.listdir()  
for f in files:  
    print(f)
```

```
# belirli dosya türlerini listeleme  
for f in files:  
    if f.endswith(".ipynb"):  
        print(f)
```

.ipynb_checkpoints
akşamGrubu.ipynb
bilgisayarlaGoru.ipynb
ders5.ipynb

akşamGrubu.ipynb
bilgisayarlaGoru.ipynb
ders5.ipynb